

Assignment 3

Разработка и интеграция инструмента оценки IoT детектора

IoT Detection Evaluation Toolkit

Аналитический отчёт и описание практического модуля

Студент: Онгарбай Нурторе

Группа: CSE-2504M

Курс: Applied Software Development Project

Astana IT University

7 октября 2026 года

Содержание и введение

1	Цель модуля и требования	3
2	Организация разработки	4
3	Выбор технологий	5
4	Системы контроля версий	6
5	Практика Git и GitHub	7
6	Непрерывная интеграция и поставка	8
7	Реализация и проверка pipeline	9
8	Особенности научного программного обеспечения	10
9	Результаты и выводы	11
	Источники и ссылки	12

Введение

Разработан инструмент оценки сохранённых бинарных предсказаний IoT-детектора. Он проверяет данные, рассчитывает метрики по сегментам и сравнивает результаты на одинаковых наблюдениях.

Практический результат включает публичный репозиторий, 49 автоматических тестов, CI на Linux и Windows и поставку версии через GitHub Release. Ниже рассматриваются основания выбора технологий, организация разработки и ограничения исследовательских выводов.

Репозиторий: [Kirun13/iiot-detection-evaluation](https://github.com/Kirun13/iiot-detection-evaluation). Проверки и публикация подтверждаются фактическими запусками от 9 октября 2026 года. Дата титульного листа обозначает дату документа.

1 Цель модуля и требования

Полный проект предполагает гибридное обнаружение атак на edge-шлюзах и федеративное обучение. Для Assignment 3 выделен небольшой самостоятельный модуль: обработка готовых предсказаний. Его цель - обеспечить проверяемый расчёт результатов эксперимента и исключить сравнение несопоставимых наборов наблюдений.

Входной CSV содержит `sample_id`, `segment`, `y_true`, `y_pred` и `model_version`. Метки 0 и 1 означают норму и атаку. Дополнительная колонка `latency_ms` содержит предоставленные измерения. В одном файле допускается одна версия модели; идентификаторы наблюдений сохраняются строками.

Таблица 1 Обязательные функции

Функция	Проверяемое условие
Проверка CSV	Колонки, метки и идентификаторы корректны; дубликаты отклоняются.
Метрики	TP, FP, TN, FN, precision, recall, F1, FPR и accuracy.
Сегменты	Сводка по всему набору и отдельно по каждому сегменту.
Сравнение	Совпадают <code>sample_id</code> , сегменты и истинные метки.
Отчёт	Создаются JSON и Markdown с версиями и SHA-256 входов.

Нефункциональные требования включают повторяемость выходных файлов, понятные сообщения об ошибках, одинаковый запуск на Linux и Windows и отдельное тестирование вычислений и файлового ввода-вывода. Ошибка входа возвращает код завершения 1; некорректный эксперимент не получает итоговый отчёт.

Модуль не обучает ML-модель, не реализует FL и не измеряет реальную задержку IDS. Он анализирует результаты, которые в дальнейшем может предоставить основной прототип. Текущие демонстрационные записи и задержки синтетические; они проверяют вычислительный инструмент.

2 Организация разработки

Работа организована как короткая итерация с фиксированным объёмом. Сначала определены CSV-контракт и ожидаемые результаты, затем реализованы вычисления, интерфейс командной строки, тесты и CI. Завершение требует работоспособного запуска, документации и доступного результата поставки.

Таблица 2 Этапы жизненного цикла

Этап	Артефакт и критерий завершения
Анализ	Границы модуля, обязательные поля, правила сравнения и ошибки.
Проектирование	Модули validation, metrics, comparison, report и cli.
Реализация	CLI выполняет обработку и сохраняет проверенный результат.
Проверка	Типизация, 49 тестов, демонстрационный запуск и CI.
Поставка	Версионный пакет, release и инструкция использования.
Сопровождение	Issues, история изменений и повторные проверки при изменениях.

Расчёты отделены от файловых операций: metrics.ts работает с типизированными наблюдениями, а cli.ts читает файлы и управляет аргументами. Это позволяет проверять научную формулу без запуска всего приложения. comparison.ts отдельно отвечает за сопоставимость экспериментов.

Definition of Done включает успешную проверку типов, тесты, README, сохранённый lock-файл и зелёный CI перед слиянием. Тесты добавлены после реализации базовых функций; строгий процесс TDD в этой работе не заявляется. Для следующей итерации новые ошибки следует сначала воспроизводить тестом.

Объём ограничен бинарной оценкой и малыми файлами. Добавление обучения модели, веб-интерфейса и потоковой обработки относится к последующим задачам. Такой выбор позволил выполнить требования разработки и интеграции без расширения до полной системы.

3 Выбор технологий

Критерии выбора - соответствие операции, сложность развёртывания, тестируемость, поддержка воспроизводимой установки и ограничение зависимостей. Для обработки готовых CSV выбран Node.js и TypeScript. Типизация проверяет структуры кода, но внешний CSV дополнительно валидируется во время выполнения [2].

Таблица 3 Выбранные технологии

Инструмент	Назначение и причина выбора
Node.js 22.19.0	CLI и файловые операции; одна версия в локальной среде и CI.
TypeScript 7.0.2	Статическая проверка Observation, Metrics и структуры отчёта.
csv-parse 7.0.3	Обработка кавычек, BOM и структуры CSV [3].
Vitest 5.0.3	Модульные проверки и полный запуск CLI [4].
npm	Установка по lock-файлу, сборка, тесты и упаковка.
GitHub Actions	Проверка проекта и поставка версионного артефакта [5].

Python был бы обоснован для обучения моделей и использования научных библиотек. В текущем модуле обучающего этапа нет, поэтому TypeScript покрывает выбранный объём. JavaScript сократил бы настройку компилятора, но не дал бы выбранной статической проверки типов. Эти альтернативы оцениваются по задаче, а не по универсальному превосходству языка.

Для CSV используется библиотека вместо разделения строки по запятой: такое разделение неверно обрабатывает кавычки и запятые внутри значений. Синхронный API выбран для простоты малой офлайн-утилиты. При больших входах потребуется потоковый API и инкрементальная агрегация.

4 Системы контроля версий

Контроль версий сохраняет историю файлов и позволяет восстановить состояние проекта. В научном ПО он связывает изменения расчётов с конкретной версией результата. Распределённая система хранит историю локально; централизованная опирается на общий сервер [6].

Таблица 4 Сравнение систем контроля версий

Система	Организация	Применимость
Git	Распределённая история; локальные коммиты и ветки.	Выбран для работы с GitHub и изоляции изменений.
Subversion	Централизованный репозиторий.	Подходит при общем серверном процессе; иная модель веток и работы без сети.
Mercurial	Распределённая история.	Альтернатива Git; потребовала бы другой платформенный workflow.

Git и платформа GitHub

Git управляет историей исходного кода. GitHub размещает Git-репозиторий и добавляет issues, pull requests, проверки и releases. Они не являются двумя конкурирующими системами одного класса. В данном проекте Git отвечает за версии, а GitHub - за публичный доступ и интеграционный процесс.

Ветка `main` хранит интегрированную версию. Изменения разрабатывались в `feature/evaluation-toolkit` и были предложены через pull request. Коммиты разделяют определение объёма, реализацию и проверки. Pull request объединяет обсуждаемый набор изменений и результаты CI.

Лицензия MIT разрешает использование и изменение кода на указанных в ней условиях. Она распространяется на авторские синтетические fixtures. Лицензия проекта не заменяет лицензию будущего внешнего датасета. Файлы `node_modules`, `dist` и временные результаты исключены из истории; готовый пакет поставляется отдельно.

5 Практика Git и GitHub

Публичный проект доступен по адресу github.com/Kirun13/iiot-detection-evaluation. README описывает формат данных, запуск, формулы, ограничения и CI/CD. Репозиторий содержит исходный код, тесты, примеры, MIT-лицензию и два workflow.

Таблица 5 Прослеживаемость разработки

Элемент	Содержание
Issue #1	Проверка входов, метрики и корректное сравнение.
Issue #2	Научные граничные случаи и воспроизводимость CLI.
Issue #3	CI, упаковка и версионная поставка.
Commit f3d0910	Фиксация объёма и правил работы.
Commit 8e51282	Вычислительный модуль и демонстрационные данные.
Commit 4601ea6	Тесты, workflows и документация.
Commit 282790b	Повторная поставка уже существующего release.
Pull request #4	Интеграция после успешных проверок на двух ОС.

Ссылка на изменения: [pull request #4](#). В его описании указаны Closes #1, Closes #2 и Closes #3; слияние закрывает связанные задачи. Статусы workflow позволяют отличить написанную конфигурацию от реально выполненной проверки.

Воспроизводимый запуск

Из чистой копии репозитория выполняются `npm ci` и `npm run verify`. Команда `verify` проверяет типы, запускает тесты, собирает код и выполняет пример сравнения. Для отдельного набора используется `npm run evaluate -- --input data/example.csv --output results`.

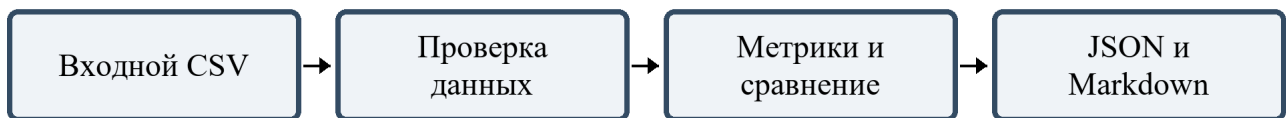
Для команды требуется Node.js версии, указанной в `.nvmrc`. Код не содержит секретов доступа к GitHub; release использует временный token workflow. По завершении проверки пользователь получает локальные `report.json` и `report.md`, а CI сохраняет их как артефакты.

6 Непрерывная интеграция и поставка

Continuous Integration (CI) означает автоматическую проверку изменений при интеграции. В проекте push и pull request запускают установку зависимостей, проверку типов, тесты и сборку. Ошибка прекращает job и делает проблему видимой до слияния [5].

Continuous Delivery (CD) готовит проверенную версию к использованию. Здесь поставляется CLI-пакет через GitHub Release; отдельного развёрнутого веб-сервиса нет. Тег v1.0.0 задаёт решение о выпуске, после которого workflow повторяет проверки и публикует артефакты. Выпуск привязан к версии Git [7].

Обработка данных



Автоматизированная проверка и поставка



Публикация по тегу выполняется после успешных проверок.

Рисунок 1 Обработка данных и автоматизированная проверка

CI выполняется на ubuntu-latest и windows-latest с одной закреплённой версией Node.js. Матрица проверяет различия файловых путей и окружения. Параметр fail-fast: false позволяет получить результаты обоих заданий даже при отказе одного.

Release проверяет совпадение имени тега с package.json. Только после успешного verify выполняется npm pack и создаётся GitHub Release. У CI есть права contents: read, у release - contents: write. Разделение прав ограничивает публикацию обычным проверочным процессом.

Пакет содержит скомпилированный dist, примеры, документацию и лицензию. JSON и Markdown публикуются как отдельные assets. Такое разделение даёт пользователю готовую утилиту и проверяемый пример результата без установки средств разработки.

7 Реализация и проверка pipeline

Конфигурации опубликованы в [.github/workflows/ci.yml](#) и [.github/workflows/release.yml](#). Pipeline был выполнен на GitHub; результат подтверждается журналами, а не только наличием YAML.

Таблица 6 Подтверждённые проверки

Проверка	Результат
Проверка типов	Успешно для исходного кода и тестов.
Модульные метрики	12 тестов прошли.
CSV-контракт	21 тест прошёл.
Сравнение наборов	7 тестов прошли.
Полный запуск CLI	9 тестов прошли.
CI Linux и Windows	Оба задания завершены успешно.
Release v1.0.0	Повторные проверки и публикация пакета успешны.

Всего подтверждено 49 тестов в четырёх файлах. CI сохраняет JUnit XML, отчёты примера и архив npm pack. Проверки zero-denominator предотвращают ложное представление неопределённой метрики как хорошего результата. Тесты сравнения исключают изменение истинных меток и наборов наблюдений.

Подтверждение CI: [запуск 37936422913](#). Подтверждение CD: [запуск 37936499531](#).
Публичная версия: [release v1.0.0](#). Даты этих запусков - 9 октября 2026 года.

При ошибке парсинга или несовпадении выборок CLI возвращает ненулевой код и не создаёт итоговый отчёт. При ошибке теста CI не публикует успешный пакет. Release также прекращается до публикации при несовпадении тега и версии. Эти условия делают поставку управляемой и проверяемой.

8 Особенности научного программного обеспечения

Научный результат требует воспроизводимости данных, среды и процедуры. Корректная программа может дать неверный вывод, если истинные метки ошибочны, `test` пересекается с `train` или сравниваются разные наблюдения. Поэтому код проверяет структуру и идентичность входов, а ограничения экспериментального протокола раскрываются отдельно.

Данные и происхождение результатов

Два fixtures содержат 12 придуманных наблюдений: по четыре для `lighting`, `pump` и `environment`, всего шесть атак и шесть нормальных примеров. Предсказания и `latency_ms` заданы вручную. Они не представляют результаты обученной модели и не содержат записи `TON_IoT`. Ожидаемые числа описаны в `data/README.md`.

В отчёт включены SHA-256 исходных байтов, `toolVersion` и `modelVersion`. Изменение файла, включая порядок строк, меняет его хеш. Сравнение допускает иной порядок строк, но проверяет те же идентификаторы, сегменты и истинные метки. Хеш устанавливает идентичность файла, но не доказывает достоверность его содержимого.

Среда и повторяемость

Node.js закреплён в `.nvmrc`, зависимости - в lock-файле, команды - в `package.json`. Отчёты не содержат текущего времени и абсолютного пути. Автоматический тест запускает утилиту дважды в разных каталогах и подтверждает одинаковые байты JSON и Markdown для одного входа.

В модуле нет случайного обучения, поэтому `seed` не требуется. Для будущего ML-этапа нужно сохранять `seed`, разбиение данных и параметры модели. Смена библиотеки или математического соглашения должна сопровождаться изменением версии и тестов, чтобы старый отчёт оставался объяснимым.

Смысл метрик

Нулевой знаменатель даёт `null` в JSON и N/A в Markdown. Бинарный F1 относится к классу «атака» и не заменяет macro-F1 по типам атак из полного проекта. `r95` использует ближайший ранг $\text{ceil}(0,95 \times n)$ в отсортированных входных измерениях. Утилита не превращает синтетическую задержку в замер оборудования.

9 Результаты и выводы

Таблица 7 Проверяемый результат синтетического примера

Показатель	Baseline	Candidate
TP / FP / TN / FN	4 / 2 / 4 / 2	5 / 1 / 5 / 1
Precision	0,6667	0,8333
Recall	0,6667	0,8333
Бинарный F1	0,6667	0,8333
FPR	0,3333	0,1667
Accuracy	0,6667	0,8333
Входной p95, мс	75	77

Разности F1 и FPR равны $+1/6$ и $-1/6$ и совпадают с ручным расчётом. По сегментам F1: environment 0,5 $\rightarrow$ 0,5; lighting 0,5 $\rightarrow$ 1; pump 1 $\rightarrow$ 1. Общая сводка дополняется оценкой каждого сегмента.

Ограничения и сложности

Загрузка CSV в память ограничивает размер набора. Внешним данным нужна валидация; сравнение требует сопоставления по ID. Неопределённые метрики должны сохранять null вместо вводящего в заблуждение числа.

Повторный выпуск тега выявил ошибку создания существующего release. Workflow исправлен: существующая версия получает обновлённые assets. Повторный запуск прошёл успешно.

Двенадцать синтетических наблюдений не подтверждают KPI реальной IDS. Обучение, FL и статистическая значимость находятся вне объёма. Дальнейшая оценка требует разрешённого test и протокола без утечки.

Заключение

Модуль разработан, проверен и опубликован. GitHub хранит задачи и версии, CI проверяет две ОС, CD поставяет пакет. Следующие задачи - подключение реального детектора, потоковая обработка и оценка типов атак с сохранением воспроизводимости.

Источники и ссылки

- [1] Assignment 3. Software Development and Integration. Предоставленное задание, с. 1.
- [2] Microsoft. The TypeScript Handbook. Статическая проверка типов и назначение TypeScript. [Открыть источник](#).
- [3] Adaltas. CSV Parse. API и обработка CSV. [Открыть источник](#).
- [4] Vitest. Getting Started. Запуск автоматических тестов. [Открыть источник](#).
- [5] GitHub Docs. Building and testing Node.js. Установка, сборка и проверки в Actions. [Открыть источник](#).
- [6] Chacon S., Straub B. Pro Git. About Version Control. Централизованные и распределённые системы. [Открыть источник](#).
- [7] GitHub Docs. About releases. Поставка версии и связь с тегами Git. [Открыть источник](#).
- [8] IoT Detection Evaluation Toolkit. Исходный код, тесты, README и CI/CD. Авторский проект. [Открыть источник](#).